

TDMA Schedule Planner and EMANE Integration

Final Technical Documentation | 16-node wireless network | 9-slot heuristic schedule

Submission status

Part 1: complete and verified. Part 2: Python-to-EMANE bridge and 16-NEM runtime validation complete, with packet-level traffic observed between NEM 1 and NEM 2.

Metric	Verified result
Nodes	16
Communication range	500 m
1-hop communication links	42
1-hop / 2-hop conflict links	90
TDMA slots	9
Reused slots	5
Maximum nodes in a slot	4
Part 1 verification	PASS
EMANE NEMs tested	16
Full schedule acceptance	16/16
Schedule rejection counters	0

1. Problem Definition and Design

The system is a centralized TDMA Schedule Planner and Optimizer. Each wireless radio is represented as a graph vertex. Two nodes form a direct communication link when their Euclidean distance is less than or equal to the configured 500 m range.

The communication graph is transformed into a Distance-2 conflict graph. Two nodes are incompatible for the same TDMA slot when their shortest-path distance in the communication graph is 1 or 2. This models the requested one-hop and two-hop interference constraints while allowing spatial reuse among nodes outside the conflict relationship.

The scheduler evaluates three greedy coloring strategies: Largest First, Smallest Last, and Saturation Largest First. A graph color represents a TDMA slot. The implementation selects the smallest slot count among the tested heuristics and then performs a local slot-reduction/improvement pass. The resulting 9-slot value is the observed heuristic result, not a proof of mathematical minimum.

Pipeline

Coordinates -> communication graph -> Distance-2 conflict graph -> graph coloring -> slot optimization -> Slot x Node matrix -> verification

2. Implementation Structure

File	Responsibility
main.py	Input handling and end-to-end execution
graph_builder.py	Communication graph and Distance-2 conflict graph construction
coloring.py	Greedy coloring heuristics and color assignment
scheduler.py	Slot normalization, optimization, reuse and matrix generation
verifier.py	Conflict-free schedule verification
generate_emanet_schedule.py	Node-to-slot schedule to native EMANE TDMA XML bridge
send_tdma_test.py	Raw Ethernet packet-level test between virtual NEM interfaces
validate_emanet.py	XML validation utility used during integration

3. Part 1 Final Result

The final clean execution used **sample_input.json** with a 500 m range. All three coloring strategies produced 9 slots, and the final verifier reported no one-hop or two-hop conflicts.

Coloring strategy	Slots
Largest First	9
Smallest Last	9
Saturation Largest First	9
Selected schedule	9

Node	Slot
Node_01	9
Node_02	5
Node_03	6
Node_04	9
Node_05	7
Node_06	1
Node_07	2
Node_08	7
Node_09	8
Node_10	3
Node_11	4
Node_12	8
Node_13	9
Node_14	5
Node_15	6
Node_16	9

4. Spatial Reuse and Slot x Node Matrix

Multiple nodes share the same slot only when they are non-conflicting in the Distance-2 conflict graph. Five slots are reused in the final schedule.

Slot	Nodes sharing the slot
5	Node_02, Node_14
6	Node_03, Node_15
7	Node_05, Node_08
8	Node_09, Node_12
9	Node_01, Node_04, Node_13, Node_16

The Slot x Node matrix uses 1 when a node is scheduled in the slot and 0 otherwise.

Slot	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16
1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0
6	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
7	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0
9	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	1

Verification rule: for every conflict edge (u,v), schedule[u] must not equal schedule[v]. The final clean Part 1 run reported PASS.

5. Part 2 EMANE Integration

The Python-to-EMANE bridge serializes the same Node-to-Slot mapping produced by Part 1 into the native EMANE TDMA schedule event format. This keeps the scheduler output as the single source of truth for the EMANE schedule.

Stage	Artifact / action
1	Part 1 Node-to-Slot mapping and schedule matrix
2	generate_emane_schedule.py
3	emane_tdma_schedule.xml
4	emaneevent-tdmaschedule publisher
5	16-NEM EMANE TDMA runtime

The generated schedule uses a 1 ms slot duration, one frame, nine slots, and 1 MHz bandwidth. The multiframe definition includes frequency 2.4G, power 0, datarate 1M, and service class 0. The final XML was validated with xmllint before publication.

Permanent bridge fix: the class=0 attribute is now generated by generate_emane_schedule.py itself, so regenerating the XML preserves the configuration required by the installed EMANE version.

6. 16-NEM Runtime Validation

Check	Observed result
EMANE process	Running one 16-NEM platform
Control port	47000 listening
Virtual interfaces	emane1 through emane16 present
Generated XML	TDMA XML VALID
Full schedule acceptance	NEM 1-16: scheduleAcceptFull = 1
Schedule rejection counters	All tested scheduleReject* counters = 0

Because the scheduler statistics are cumulative, a repeated publication can increment scheduleAcceptFull. The final clean run cleared the statistics before the single final publication, producing scheduleAcceptFull = 1 for each NEM.

7. Packet-Level EMANE Test

A raw Ethernet test generated 50 frames from the virtual interface attached to NEM 1 toward NEM 2. A 10 dB pathloss event was configured between NEM 1 and NEM 2 so the precomputed propagation model had an explicit usable path.

Measurement	Final observed value
Test frames generated	50
NEM 2 processedUpstreamPackets	50
NEM 2 numRxSlotValid	64
NEM 2 virtual interface RX packets	64
NEM 2 scheduleAcceptFull	1
NEM 2 scheduleRejectOther	0

Interpretation: the final run confirms packet traffic was processed by the NEM 2 TDMA MAC and that receive slots were observed while the schedule remained accepted. The interface and valid-slot counters are cumulative and therefore are not interpreted as a 50/50 packet-delivery percentage. A controlled allow-slot versus deny-slot experiment was not used to claim a delivery ratio.

8. Test Cases

Test	Purpose	Observed result
Chain test	Three-node line to exercise two-hop conflict handling	PASS
Spatial reuse test	Isolated/non-conflicting node reuse	PASS
Boundary test	Exactly 500 m treated as a communication link	PASS
16-node final run	Full scheduler execution and verification	PASS
16-NEM EMANE schedule test	Acceptance of generated schedule on all NEMs	PASS
NEM 1 -> NEM 2 packet test	Observe packet flow through EMANE TDMA path	Observed traffic / valid RX slots

9. Submission and Defense Checklist

Item	Status
Part 1 source code	Complete
Part 1 final execution evidence	Part1_Final_Run.txt
Part 2 bridge source	Complete
EMANE schedule XML	Complete
16-NEM platform configuration	Complete
Part 2 runtime evidence	Part2_Final_Evidence.txt
README	Updated
Documentation PDF	This document
Presentation PPT	Final technical presentation

Be prepared to explain: why Distance-2 conflicts are needed; the difference between communication and conflict graphs; why coloring is heuristic; how slot reuse works; how the Node-to-Slot mapping is serialized into EMANE; what scheduleAcceptFull and scheduleReject* mean; and what the packet counters do and do not prove.

10. Conclusion

The completed implementation provides a centralized graph-based TDMA schedule planner for a 16-node wireless network. The clean Part 1 execution generates a verified 9-slot schedule with five reused slots and no modeled one-hop or two-hop conflicts.

The Part 2 bridge now converts the same schedule directly into native EMANE TDMA XML, including the required class=0 field. A clean 16-NEM EMANE runtime accepted the regenerated schedule on all 16 NEMs with zero tested schedule rejection counters. A packet-level NEM 1 to NEM 2 test also observed traffic and valid receive slots.

The submission deliberately avoids claiming a mathematical optimum or a 100 percent packet-delivery rate. The measured results reported here are limited to what was directly observed in the implementation and EMANE runtime.